

Unified SOC Command Dashboard - Installation, Runtime Permissions, and Use

Date: 2026-07-15

Repository: Q-CFoulon/BP-API-integration

Software: Unified SOC Command Dashboard / BP API Integration

Executive Summary

The Unified SOC Command Dashboard is a local-first security operations dashboard that combines Blackpoint Cyber CompassOne data with Microsoft Defender XDR incident data. It is used by SOC analysts to view MDR detections, inspect Defender XDR incidents, correlate alerts across systems, manage tenant onboarding, record case closeout decisions, and optionally dispatch approved remediation actions to a separately deployed Defender Response MCP gateway.

The application does not require local administrator rights to run once Node.js is installed. It does require controlled access to security APIs, API keys, Entra app registrations, and outbound HTTPS connectivity to Blackpoint and Microsoft services.

What The Software Is Used For

Capability	Purpose
Blackpoint CompassOne integration	Lists managed tenants, detections, alerts, analytics, reports, and asset inventory from Blackpoint CompassOne.
Microsoft Defender XDR integration	Lists Defender incidents, retrieves incident details and related alert links, and writes approved incident updates such as assignment, status, classification, determination, and tags.
Cross-source correlation	Helps analysts connect Blackpoint detections to Defender XDR incidents using tenant, entity, time, and incident context.
Tenant onboarding	Lets an operator add or update tenant entries without manually editing JSON for every onboarding step.
Triage and remediation workflow	Produces remediation proposals and supports human approval before any automated response is dispatched.
Closeout governance	Captures resolution taxonomy, closeout notes, and audit context for case handling.
Audit and tracking	Records analyst actions and local workflow state, depending on the configured storage backend.

Installation Permissions

Local Workstation Requirements

Requirement	Permission Needed	Why It Is Needed
Node.js 20 or later	Local software installation rights if Node.js is not already installed.	Runs the Vite frontend, Express backend, TypeScript tooling, and build scripts.
npm 9 or later	Included with Node.js; internet access to npm registry for dependency install.	Installs JavaScript dependencies from package.json.
Git or repository access	Read access to the repository and write access to the local checkout folder.	Clones or updates the application source code.
Local filesystem write access	Write access to the project directory.	Creates node_modules, .env, config/tenants.json, build output, logs, and local generated documents.
Local ports	Ability to bind localhost ports 3000 and 3001 by default.	3000 serves the Vite UI; 3001 serves the backend API.
Optional Docker	Docker Desktop / container runtime rights if using the Docker deployment path.	Builds and runs the application container instead of running directly with npm.

Local administrator rights are normally only needed to install system software such as Node.js, Git, Docker Desktop, or corporate certificate tooling. Daily use of the application should run as a standard user.

Install Commands

```
npm install
npm run dev
```

For production-style local validation:

```
npm run build
npm start
```

Runtime Network Access

Destination	Direction	Port/Protocol	Used For
http://localhost:3000	Local inbound	HTTP	Development frontend UI.
http://localhost:3001	Local inbound	HTTP	Development backend API and health endpoint.
https://api.blackpointcyber.com	Outbound	HTTPS 443	Blackpoint CompassOne API.
https://login.microsoftonline.com	Outbound	HTTPS 443	Entra ID user sign-in, JWKS token validation, and client credential token acquisition.
https://graph.microsoft.com	Outbound	HTTPS 443	Microsoft Graph Security API for Defender XDR incidents and alerts.
https://api.security.microsoft.com	Outbound	HTTPS 443	Optional Defender security API host when configured per tenant.
Configured MCP webhook URL	Outbound	HTTPS 443 recommended	Optional dispatch of approved remediation proposals to a separate automation gateway.
https://registry.npmjs.org	Outbound	HTTPS 443	Dependency installation with npm.

GitHub or internal Git remote	Outbound	HTTPS 443 or SSH 22	Source checkout and updates.
-------------------------------	----------	---------------------	------------------------------

Production deployments should use HTTPS, a managed reverse proxy or hosting platform, and network controls appropriate for SOC tooling.

Application Configuration Secrets

Secret / Setting	Location	Purpose	Handling Requirement
COMPASSONE_API_KEY	.env	Account-level Blackpoint CompassOne API bearer token.	Store only on backend/server side. Do not expose to frontend variables or commit to Git.
COMPASSONE_API_URL	.env	CompassOne API base URL, usually https://api.blackpointcyber.com .	Non-secret but environment specific.
blackpoint.customerId	config/tenants.json	Blackpoint tenant/customer UUID used as x-tenant-id.	Treat as customer metadata; keep out of public repos.
blackpoint.apiKeyOverride	config/tenants.json or env reference	Optional tenant-specific Blackpoint key.	Prefer \${ENV_VAR} reference instead of literal key.
Microsoft tenantId	config/tenants.json	Customer Entra tenant ID.	Customer metadata; restrict sharing.
Microsoft clientId	config/tenants.json	Per-tenant service app registration client ID.	Non-secret identifier, but still operationally sensitive.
Microsoft clientSecret	.env via \${ENV_VAR} reference	Per-tenant service app secret for client credential flow.	Secret. Store in .env, Key Vault, or managed secret store; rotate regularly.
VITE_ENTRA_CLIENT_ID	.env / build env	Frontend SPA app registration client ID for user sign-in.	Non-secret public client ID.
VITE_ENTRA_TENANT_ID	.env / build env	Platform Entra tenant ID for user sign-in.	Non-secret identifier.
ENTRA_TENANT_ID	.env / runtime env	Backend token issuer tenant for API auth validation.	Non-secret identifier.
ENTRA_CLIENT_ID	.env / runtime env	Backend API audience/client ID.	Non-secret identifier.
MCP_AUTOMATION_WEBHOOK_SECRET	.env	Optional shared secret for signed remediation webhook dispatch.	Secret. Rotate if exposed.

Storage credentials	.env	Optional PostgreSQL or Cosmos DB connection settings.	Secret. Use managed identity or secret store where possible.
---------------------	------	---	--

Blackpoint CompassOne Permissions

The dashboard uses a Blackpoint CompassOne API bearer token and sends the customer UUID as the x-tenant-id header for tenant-scoped requests.

Minimum Blackpoint access should allow read-only access to the following CompassOne API areas:

CompassOne Area	API Use In This App	Access Needed
Tenants	GET /v1/tenants for onboarding tenant selection.	Account-level tenant listing permission.
Alert groups / detections	GET /v1/alert-groups, individual alert group lookup, counts, weekly trends, top entities, top threats.	Tenant-scoped read permission for detections / alert groups.
Alerts	Alerts inside a detection / alert group.	Tenant-scoped read permission for alerts.
Reports	List reports, get report PDF URL, get report JSON payload.	Tenant-scoped report read permission.
Assets	List assets and count assets.	Tenant-scoped asset inventory read permission.

Known Blackpoint considerations:

- The API key is passed as Authorization: Bearer {api_key}.
- Tenant-scoped calls require x-tenant-id: {customerId}.
- Some CompassOne endpoints are role-limited and may return 403 until Blackpoint grants the correct API role.
- This application currently reads Blackpoint data; it does not require Blackpoint write or administrative privileges for the implemented routes.

Microsoft Entra And Defender XDR Permissions

The application uses two Microsoft identity patterns:

1. A platform/user sign-in app registration for the dashboard UI and backend API authentication.
2. Per-customer confidential client app registrations for Defender XDR / Microsoft Graph Security API access.

1. Platform User Sign-In App Registration

Used by the React SPA and backend middleware to authenticate SOC users.

Permission / Setting	Requirement
App type	Single-page application / public client using authorization code with PKCE.
Redirect URI	Local development: http://localhost:3000 or the configured Vite origin. Production: the deployed HTTPS URL.

Exposed API scope	api://<client-id>/access_as_user or the matching backend API scope configured for the app.
Backend validation	Backend validates bearer tokens using ENTRA_TENANT_ID, ENTRA_CLIENT_ID, issuer, audience, and Entra JWKS keys.
User assignment	SOC users should be assigned to the app through Entra ID.
App roles	Recommended roles are SOC.Admin, SOC.Analyst, and SOC.Viewer to support role-based access controls.
Conditional Access	Recommended for production: MFA, compliant device, trusted location, and session controls aligned to SOC policy.

2. Per-Tenant Defender XDR Service App Registration

Used by the backend through MSAL confidential client flow. Each Microsoft-enabled tenant entry has a tenant ID, client ID, and client secret. The backend requests the `https://graph.microsoft.com/.default` scope or the configured security API host `.default` scope.

Recommended Microsoft Graph application permissions:

Permission	Type	Why It Is Needed
SecurityIncident.Read.All	Application	Read Defender XDR incidents from Microsoft Graph Security API.
SecurityIncident.ReadWrite.All	Application	Update incident assignment, status, classification, determination, and tags. Required for writeback.
SecurityAlert.Read.All	Application	Read related alert information and construct evidence/deep links.

Administrative consent is required in each customer tenant for application permissions. If the application is configured to use `https://api.security.microsoft.com` instead of Microsoft Graph, grant the equivalent Microsoft Defender XDR API application permissions for incident read and incident read/write.

Microsoft prerequisites:

- Microsoft Defender XDR must be available and licensed for the tenant.
- The service principal should be restricted to the minimum permissions needed for the enabled workloads.
- Client secrets should be stored outside source control and rotated on a defined schedule.
- Production deployments should prefer certificate credentials or managed identity patterns where possible.

Optional Defender Response MCP Gateway Permissions

The dashboard can send approved remediation proposals to a separately deployed Defender Response MCP gateway by using `MCP_AUTOMATION_WEBHOOK_URL` and `MCP_AUTOMATION_WEBHOOK_SECRET`.

The dashboard itself does not need direct device isolation, account disablement, password reset, or session revocation permissions unless those actions are implemented locally. Those privileged permissions belong to the separate automation gateway and should be documented and approved separately.

Typical gateway-level permissions may include highly privileged Defender for Endpoint, Microsoft Graph, or Entra permissions for actions such as device isolation, AV scan, account disablement, session revocation, or password reset. Use a human approval workflow and least privilege for each action.

Storage Permissions

Storage Mode	Permission Needed	Notes
Memory	None beyond local process memory.	Default development mode. Data is lost when the process stops.
PostgreSQL	Database account with permission to connect and read/write application tables.	Use TLS and a least-privilege database user.
Azure Cosmos DB	Cosmos DB key or identity with read/write access to the application database/container.	Prefer managed identity/RBAC over account keys in production.

Operator Permissions

Operator Role	Typical Access
Viewer	Read dashboards, detections, incidents, reports, and tenant summaries.
Analyst	Viewer access plus triage notes, case updates, correlation actions, closeout preparation, and remediation proposal review.
Admin	Analyst access plus tenant onboarding, tenant configuration updates, integration configuration, and role management.

Production environments should enforce user authentication and RBAC. Administrative functions such as tenant onboarding and integration changes should be limited to approved SOC administrators.

Data Handled By The Software

The application may process or display:

- Customer and tenant identifiers.
- Blackpoint detections, alerts, assets, reports, and analytics.
- Defender XDR incidents, alert references, severity, status, classifications, determinations, tags, and assignment data.
- Analyst workflow data such as triage, remediation proposals, approvals/rejections, closeout notes, and audit events.
- API credentials and client secrets on the backend only.

Because this is security operations data, treat application access as privileged even when the current deployment is local.

Least-Privilege Summary

Minimum permissions for a typical local development or pilot deployment:

1. Standard local user rights to run Node.js and bind localhost ports.
2. Write access to the local project folder for dependencies and local config files.
3. Blackpoint CompassOne API key with read access to tenants, detections, alerts, reports, and assets.
4. Entra SPA/API app registration for SOC user sign-in.
5. Per-tenant Microsoft confidential client app with admin-consented application permissions for Defender incident read and, if writeback is enabled, incident read/write.
6. Outbound HTTPS access to Blackpoint, Microsoft identity, Microsoft Graph/Security APIs, npm registry, and source control.
7. Optional storage credentials only if using PostgreSQL or Cosmos DB instead of memory storage.
8. Optional MCP webhook secret and gateway permissions only if automated remediation dispatch is enabled.

Approval Checklist

- ☐ Node.js and npm installation approved.
- ☐ Repository access approved.
- ☐ Local ports 3000 and 3001 approved for development.
- ☐ Outbound HTTPS destinations approved by network/security team.
- ☐ Blackpoint API key scope reviewed and approved.
- ☐ Entra user sign-in app registration configured.
- ☐ Per-tenant Defender XDR service app registration configured.
- ☐ Microsoft Graph / Defender permissions admin-consented.
- ☐ Secrets stored outside Git and rotation owner assigned.
- ☐ Optional storage access reviewed.
- ☐ Optional remediation gateway permissions reviewed separately.
- ☐ RBAC roles and operator access reviewed.

